

# Cahier des charges

Conception et développement d'un  
Application E-commerce Basé sur  
une architecture Micro service

Réaliser par : Abada Aziz & Mohamed Jajaa

Encadré par : NAINIA Omar

## Tableau de matière :

|                                                                |    |
|----------------------------------------------------------------|----|
| Tableau de matière :                                           | 2  |
| <b>1- Le contexte :</b>                                        | 4  |
| <b>2- Objectif du projet :</b>                                 | 4  |
| <b>3- Éléments Fonctionnels :</b>                              | 5  |
| 3.1- Problématique :                                           | 5  |
| 3.2- Périmètre Fonctionnels :                                  | 5  |
| 3.2.1- Module Authentification (auth-service) :                | 5  |
| 3.2.2- Module Catalogue Produits (product-service) :           | 5  |
| 3.2.3- Module Commandes (order-service) :                      | 6  |
| 3.2.4- Interface Utilisateur (Frontend React) :                | 6  |
| 3.2.5- Navigation :                                            | 6  |
| 3.2.6- organigramme fonctionnel :                              | 7  |
| <b>4- Éléments Techniques :</b>                                | 8  |
| 4.1- Architecture (Micro Services) :                           | 8  |
| 4.2- Choix technologiques et justification :                   | 9  |
| <b>5- Référentiels juridiques, qualitatifs et empiriques :</b> | 11 |
| 5.1- Référentiels qualitatifs :                                | 11 |
| 5.2- Référentiels juridiques :                                 | 11 |
| 5.3- Référentiels empiriques / outils d'évaluation :           | 12 |
| <b>6- Organisation :</b>                                       | 12 |
| 6.1- Méthodologie Agile (Scrum) :                              | 12 |
| 6.1.1- Planning prévisionnel (exemple) :                       | 12 |
| 6.1.2- Ordonnancement (Diagramme de Gantt) :                   | 13 |
| 6.1.6 Rôles dans l'équipe Scrum :                              | 13 |
| 6.2- Outils utilisés :                                         | 13 |
| <b>7- Éléments juridiques :</b>                                | 14 |
| 7.1- Instance d'arbitrage :                                    | 14 |
| 7.2- Pénalités de retard (simulées) :                          | 14 |
| <b>8- Conception et Modélisation :</b>                         | 14 |
| 8.1- Modélisation :                                            | 14 |
| 8.1.1- Dictionnaire de Données (DD) :                          | 14 |
| 8.1.2- Modèle Conceptuel de Données (MCD) :                    | 17 |
| 8.1.3- Le schéma relationnel de Données :                      | 17 |
| 8.1.4- Diagramme de use case :                                 | 18 |

|                                                                  |    |
|------------------------------------------------------------------|----|
| 8.1.5 Diagramme de séquence – Ajouter au panier : .....          | 19 |
| 8.1.6 Diagramme de séquence – Passer une commande : .....        | 20 |
| 8.1.7 Diagramme de séquence – Consulter les commandes : .....    | 21 |
| 8.1.8 Diagramme de séquence – Gérer les produits (Admin) : ..... | 22 |
| 8.1.9 Diagramme de classe : .....                                | 23 |
| 8.2- Modélisation : .....                                        | 24 |
| 8.2.1- page S'inscrire .....                                     | 24 |
| 8.2.2- page connexion .....                                      | 24 |
| 8.2.3- page des produits .....                                   | 25 |
| 8.2.4- page connexion .....                                      | 26 |

# 1-Le contexte :

Ce projet s'inscrit dans le cadre de nos formations en développement digital – option Web Full Stack, dispensée par ISTA NTIC Safi. Il s'agit d'un projet de synthèse visant à mettre en pratique les compétences acquises tout au long de la formation, notamment en matière de développement d'applications web en architecture microservices, de gestion d'authentification sécurisée, de conception d'interfaces modernes en React.js et d'utilisation de conteneurs Docker pour le déploiement.

Le projet n'est pas commandité par un client réel, mais simule un cas concret de digitalisation d'une plateforme e-commerce pour permettre aux étudiants de développer une solution complète, modulaire et sécurisée, intégrant les meilleures pratiques du développement moderne.

Il représente également une opportunité de mise en œuvre de méthodologies de travail en équipe (Scrum/Agile), de versioning avec Git, de documentation technique, et de planification d'un projet logiciel du cahier des charges jusqu'au déploiement.

# 2-Objectif du projet :

L'objectif principal du projet est de concevoir et développer une application e-commerce complète en utilisant une architecture de microservices.

L'application doit permettre aux utilisateurs de :

- Créer un compte et s'authentifier de manière sécurisée,
- Consulter un catalogue de produits,
- Passer des commandes,
- Visualiser l'historique de leurs achats.

Le projet vise également à :

- **Appliquer les principes de l'architecture microservices**, avec des services indépendants pour l'authentification, les produits et les commandes,
- **Déployer les services via Docker**, pour simuler un environnement de production modulaire,
- **Concevoir une interface frontend moderne avec React.js**, en assurant une bonne expérience utilisateur,
- **Assurer la sécurité des échanges** via l'authentification JWT et les bonnes pratiques de validation côté backend.

## 3-Éléments Fonctionnels :

### 3.1- Problématique :

Avec la généralisation du commerce en ligne, les entreprises cherchent à adopter des architectures logicielles plus flexibles, modulaires et évolutives pour répondre efficacement aux besoins des utilisateurs.

La plupart des plateformes e-commerce classiques sont construites en architecture monolithique, ce qui complique leur maintenance, leur scalabilité et l'ajout de nouvelles fonctionnalités.

De plus, dans le cadre d'une formation en développement web full stack, il est essentiel de savoir concevoir des applications modernes, sécurisées et déployables, en intégrant de multiples technologies (API REST, authentification JWT, conteneurs Docker, React.js, MongoDB...).

Ce projet répond donc à la problématique suivante :

Comment concevoir et développer une plateforme e-commerce modulaire et sécurisée, basée sur une architecture de microservices, tout en mobilisant les compétences full stack acquises durant la formation ?

Souhaites-tu que je reformule cette problématique de façon plus académique, plus technique ou plus concise selon ton style préféré ? On peut aussi l'intégrer dans une introduction globale si tu veux une version formelle.

### 3.2- Périmètre Fonctionnels :

L'application e-commerce devra offrir les fonctionnalités suivantes, organisées par **rôles** et **modules fonctionnels** :

#### 3.2.1- Module Authentification (auth-service)

- Inscription d'un nouvel utilisateur.
- Connexion de l'utilisateur via email et mot de passe.
- Gestion sécurisée des sessions avec JWT.
- Déconnexion.
- Accès restreint aux fonctionnalités selon l'état de connexion.

#### 3.2.2- Module Catalogue Produits (product-service)

- Affichage de la liste des produits disponibles (nom, image, description, prix, stock).
- Recherche de produits par mot-clé.
- Affichage des détails d'un produit.
- (Admin) Ajout, modification et suppression de produits (CRUD produits).
- Vérification du stock disponible avant ajout au panier ou commande.

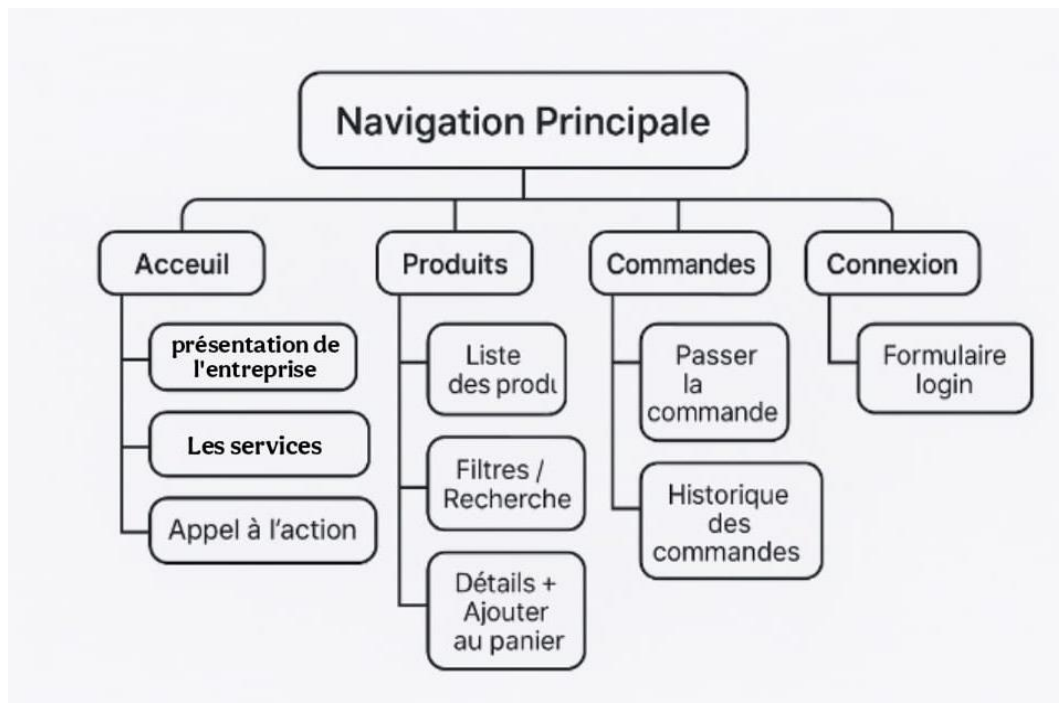
### 3.2.3- Module Commandes (order-service)

- Ajout d'un produit au panier.
- Validation et création d'une commande.
- Enregistrement des détails de la commande (produits, quantités, prix, date).
- Consultation de l'historique des commandes de l'utilisateur connecté.

### 3.2.4- Interface Utilisateur (Frontend React)

- Interface moderne et responsive.
- Navigation entre les pages.
- Formulaire ergonomiques pour l'authentification et la commande.
- Gestion des erreurs (ex : produit indisponible, connexion échouée).

### 3.2.5- Navigation



L'arborescence du menu principal d'un site e-commerce. Il montre comment les différentes rubriques sont organisées pour offrir une navigation fluide et intuitive aux utilisateurs.

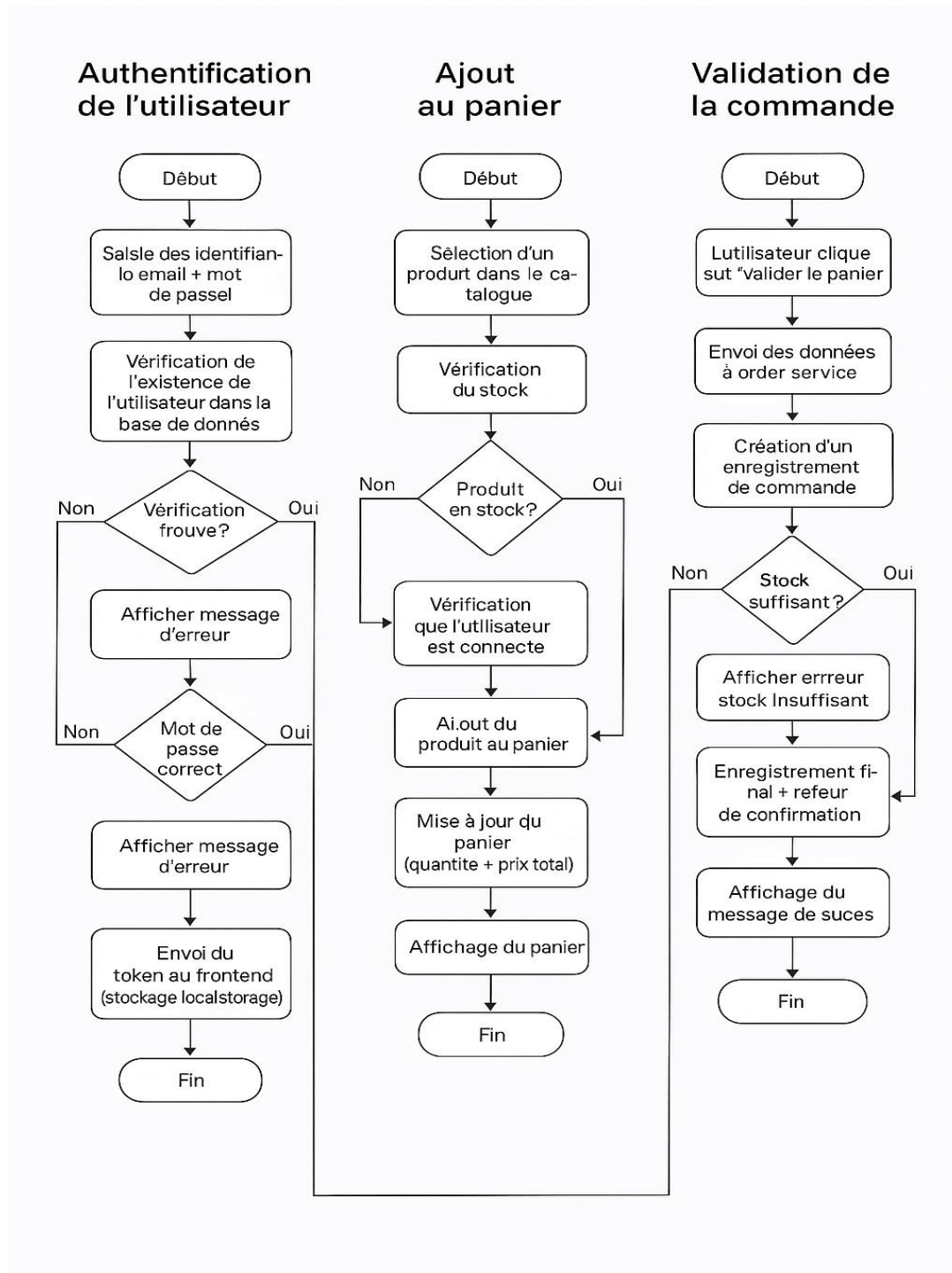
#### Rubriques principales :

- **Accueil** : Présente l'entreprise, ses services et un appel à l'action pour inciter les visiteurs à découvrir les produits.
- **Produits** : Permet aux clients de consulter la liste des produits, de filtrer les résultats, de rechercher des articles et de voir les détails avant de les ajouter au panier.
- **Commandes** : Regroupe la gestion du panier, la validation de commande et l'historique des achats.
- **Connexion / Inscription** : Accès aux formulaires pour se connecter ou créer un compte utilisateur.

## Objectif :

Améliorer l'expérience utilisateur en facilitant la navigation entre les pages essentielles du site.

### 3.2.6- organigramme fonctionnel



## 4-Éléments Techniques

### 4.1- Architecture (Micro Services) :

#### ❖ Pourquoi avoir choisi l'architecture microservices ?

L'architecture microservices a été choisie pour ce projet e-commerce afin de répondre à plusieurs objectifs pédagogiques et techniques :

- **Modularité** : Chaque fonctionnalité principale (authentification, produits, commandes) est isolée dans un service indépendant, ce qui facilite la compréhension et la maintenance.
- **Scalabilité** : Les services peuvent être mis à l'échelle individuellement selon la charge (ex : le catalogue produit peut avoir plus de trafic que le service de commande).
- **Indépendance technologique** : Chaque microservice peut évoluer séparément, utiliser une base de données dédiée, ou être mis à jour sans impacter les autres.
- **Meilleure résilience** : Un bug dans un service (ex. : produit) ne bloque pas l'ensemble de l'application.
- **Expérience pratique** : Cela permet d'appliquer les bonnes pratiques DevOps, Docker, et la gestion des API REST dans un contexte concret, comme en entreprise.

#### ❖ Comment cette architecture est appliquée dans le projet ?

L'architecture est structurée autour de trois microservices principaux et d'un frontend unique, chacun étant déployé dans un conteneur Docker. Voici la répartition :

##### **1. auth-service**

- ✓ Gère l'inscription, la connexion, la génération de JWT, la validation des utilisateurs.
- ✓ Base de données MongoDB dédiée aux utilisateurs.
- ✓ Routes protégées, middleware de vérification du token.

##### **2. product-service**

- ✓ Gère l'ajout, la modification, la suppression et la consultation des produits.
- ✓ Chaque produit contient : nom, description, prix, image, stock.
- ✓ Base MongoDB dédiée aux produits.
- ✓ Exposition d'une API REST pour consultation par le frontend.

##### **3. order-service**

- ✓ Gère les paniers, la création et la consultation des commandes.
- ✓ Stocke l'historique des commandes pour chaque utilisateur.
- ✓ Vérifie le stock via des appels inter-services (ou via un event bus dans une version avancée).

##### **4. frontend**

- ✓ Interface utilisateur unique qui interagit avec les 3 services via des appels Axios (API REST).
- ✓ Présente les écrans : inscription, connexion, liste des produits, panier, commandes.

#### Déploiement technique

- ✓ Chaque service est contenu dans un dossier distinct avec son propre package.json et ses dépendances.
- ✓ L'orchestration des services se fait via Docker Compose (docker-compose.yml) pour les lancer ensemble avec leurs bases de données.

## 4.2- Choix technologiques et justification

Pour assurer la cohérence, la simplicité de développement et l'adéquation aux besoins d'une application e-commerce en microservices, les technologies suivantes ont été choisies :

### Node.js & Express.js (pour tous les micro services)



#### Pourquoi Node.js ?

- Node.js est **asynchrone, rapide et léger**, idéal pour des API performantes.
- Très adapté aux **architectures REST** avec gestion efficace des entrées/sorties.
- Utilise JavaScript côté serveur, ce qui permet d'unifier le langage sur tout le projet (frontend + backend).

#### Pourquoi Express.js ?

- Framework minimaliste pour Node.js, facile à configurer.
- Grande communauté, très bien documenté.
- Permet de créer rapidement des routes RESTful sécurisées et maintenables.

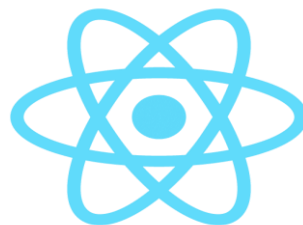
## MongoDB (pour chaque service)



### Pourquoi MongoDB ?

- Base de données **NoSQL** orientée documents (JSON-like) très adaptée aux microservices.
- Flexible pour stocker des données non structurées comme les utilisateurs, produits ou commandes.
- Performante et facile à intégrer avec Node.js grâce à la bibliothèque Mongoose.
- Chaque service ayant sa propre base MongoDB favorise le découplage et la résilience.

## React.js (pour le frontend)



### Pourquoi React ?

- React est une **bibliothèque moderne** de création d'interfaces dynamiques et réactives.
- Basé sur les **composants**, facilitant la réutilisation du code et la gestion des états.
- Compatible avec les appels API REST (via Axios ou Fetch).
- Permet de concevoir une UI fluide, responsive et agréable à utiliser.

## JWT (JSON Web Token)



### Pourquoi JWT ?

- Méthode sécurisée, légère et standardisée pour gérer les sessions utilisateur sans stockage côté serveur.
- Le token est stocké côté client et envoyé dans les headers, évitant le recours à des sessions serveur.

- Facile à implémenter dans un microservice auth-service pour protéger les routes des autres services.

## Docker & Docker Compose



### Pourquoi Docker ?

- Permet de **conteneuriser chaque microservice** avec ses dépendances.
- Assure un **environnement cohérent** pour le développement, les tests et la production.
- Simplifie le déploiement en regroupant tous les services dans un fichier docker-compose.yml.
- Favorise l'apprentissage de bonnes pratiques DevOps (portabilité, isolation, scalabilité).

## 5-Référentiels juridiques, qualitatifs et empiriques

Même s'il s'agit d'un projet académique, la conception et la réalisation de l'application s'inspirent des bonnes pratiques professionnelles en matière de développement logiciel, de qualité, et de conformité.

### 5.1- Référentiels qualitatifs

- **Norme ISO 25010** (qualité des logiciels) :  
L'application vise à respecter les principales caractéristiques définies par cette norme :
  - ✓ **Fonctionnalité** : couverture complète des besoins fonctionnels (inscription, commande, catalogue...).
  - ✓ **Fiabilité** : microservices isolés et robustes, tolérance aux erreurs.
  - ✓ **Performance** : chargement rapide, traitement efficace des commandes.
  - ✓ **Sécurité** : authentification sécurisée via JWT, validation des données.
  - ✓ **Maintenabilité** : code structuré, services indépendants, commentaires et documentation.
  - ✓ **Portabilité** : conteneurisation via Docker.

### 5.2- Référentiels juridiques

Même si le projet n'est pas déployé à des fins commerciales, certaines notions légales sont Prises en compte :

- **RGPD** (Règlement Général sur la Protection des Données) :
  - Les données des utilisateurs sont traitées avec précaution.
  - Aucune donnée sensible n'est exploitée sans consentement.
  - Les mots de passe sont stockés de manière sécurisée (hash).

- **Conditions d'utilisation (simulées) :**
  - L'utilisateur accepte que ses données soient utilisées dans le cadre de l'application.
  - La suppression d'un compte ou des données personnelles est prévue dans une version évoluée.

### 5.3- Référentiels empiriques / outils d'évaluation

- Utilisation de **SonarQube (optionnel)** pour évaluer la qualité du code (maintenabilité, duplication, bugs).
- Tests manuels ou automatisés pour assurer la conformité aux cas d'utilisation définis.
- Documentation et vérification systématique des API (ex. via Postman ou Swagger).

## 6-Organisation :

### 6.1- Méthodologie Agile (Scrum) :

Le projet sera réalisé selon la méthodologie Agile Scrum, qui permet un développement itératif, incrémental et collaboratif.

#### ➤ Rythme de travail :

- Durée totale : à préciser : **5 à 7 semaines**
- Découpé en **sprints** de 1 ou 2 semaines.

#### 6.1.1- Planning prévisionnel (exemple) :

| Semaine | Sprint    | Objectifs                                                       |
|---------|-----------|-----------------------------------------------------------------|
| S1      | Sprint 1  | Cahier des charges Fonctionnel                                  |
| S2      | Sprint 2  | Mise en place les interfaces graphique (React js)               |
| S3      | Sprint 3  | Mise en place les interfaces graphique (React js)               |
| S4      | Sprint 4  | Mise en place la partie Backend (Auth-service, product-service) |
| S5      | Sprint 5  | Mise en place la partie Backend (Order-service)                 |
| S6      | Sprint 6  | Lancer l'application et déployer chez docker                    |
| S7      | Livraison | Présentation + documentation (Rapport)                          |

### 6.1.2- Ordonnancement (Diagramme de Gantt)

| Les sprints/les semaines | Semaine 1 | Semaine 2 | Semaine 3 | Semaine 4 | Semaine 5 | Semaine 6 | Semaine 7 |
|--------------------------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| S1                       |           |           |           |           |           |           |           |
| S2                       |           |           |           |           |           |           |           |
| S3                       |           |           |           |           |           |           |           |
| S4                       |           |           |           |           |           |           |           |
| S5                       |           |           |           |           |           |           |           |
| S6                       |           |           |           |           |           |           |           |
| S6                       |           |           |           |           |           |           |           |

**Chemin critique** = S1 → S2 → S3 → S4 → S5 → S6

Tout retard sur l'un de ces sprints retardera automatiquement la livraison finale du projet.

### 6.1.6 Rôles dans l'équipe Scrum

| Rôle          | Personne assignée (simulation) |
|---------------|--------------------------------|
| Product Owner | ABADA AZIZ                     |
| Scrum Master  | JAJAA MOHAMED                  |
| Développeurs  | ABADA AZIZ & VJAJAA MOHAMED    |

Bien que chaque membre de l'équipe ait un rôle défini selon le cadre Scrum, une flexibilité a été maintenue tout au long du développement.

Cela signifie que chaque membre a pu intervenir en soutien sur d'autres tâches, selon la charge, les compétences et les besoins du sprint.

L'objectif était de favoriser l'efficacité, l'entraide et la continuité du développement, tout en respectant l'esprit Agile.

### 6.2- Outils utilisés

| Outil                      | Usage principal                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Git + GitHub               | Versioning du code source, collaboration en équipe                                                                               |
| JIRA                       | Suivi des tâches, organisation des sprints, backlog produit                                                                      |
| Postman                    | Test et documentation des API REST (routes, authentification...)                                                                 |
| Discord / WhatsApp / Email | Communication au sein de l'équipe projet                                                                                         |
| SonarQube                  | Analyse statique du code : détection d'erreurs, duplications, dette technique, respect des bonnes pratiques (qualité logicielle) |

## 7-Éléments juridiques :

Bien que ce projet soit réalisé dans un cadre académique, cette section simule les dispositions juridiques qui seraient en vigueur dans un projet professionnel impliquant un client réel et un prestataire.

### 7.1- Instance d'arbitrage

En cas de désaccord, de non-respect des livrables ou de divergence d'interprétation des besoins :

- L'instance d'arbitrage est représentée par le formateur référent (ou le jury pédagogique), qui joue le rôle de médiateur entre le "maître d'ouvrage" et l'équipe projet.
- Toute décision prise par cette instance est considérée comme définitive dans le cadre du projet.

### 7.2- Pénalités de retard (simulées)

Afin de respecter les engagements en termes de délai (planning de sprints), des pénalités pédagogiques peuvent être envisagées en cas de retard non justifié :

- Un retard sur une livraison de sprint peut entraîner :
  - Une note réduite pour l'étape concernée,
  - Une présentation reportée avec perte de points,
  - Un signalement pédagogique en cas de non-livraison totale.

Cette disposition simule les clauses contractuelles habituelles dans les projets informatiques réels, dans lesquels des pénalités financières ou juridiques sont prévues en cas de non-respect du planning.

## 8-Conception et Modélisation :

### 8.1- Modélisation :

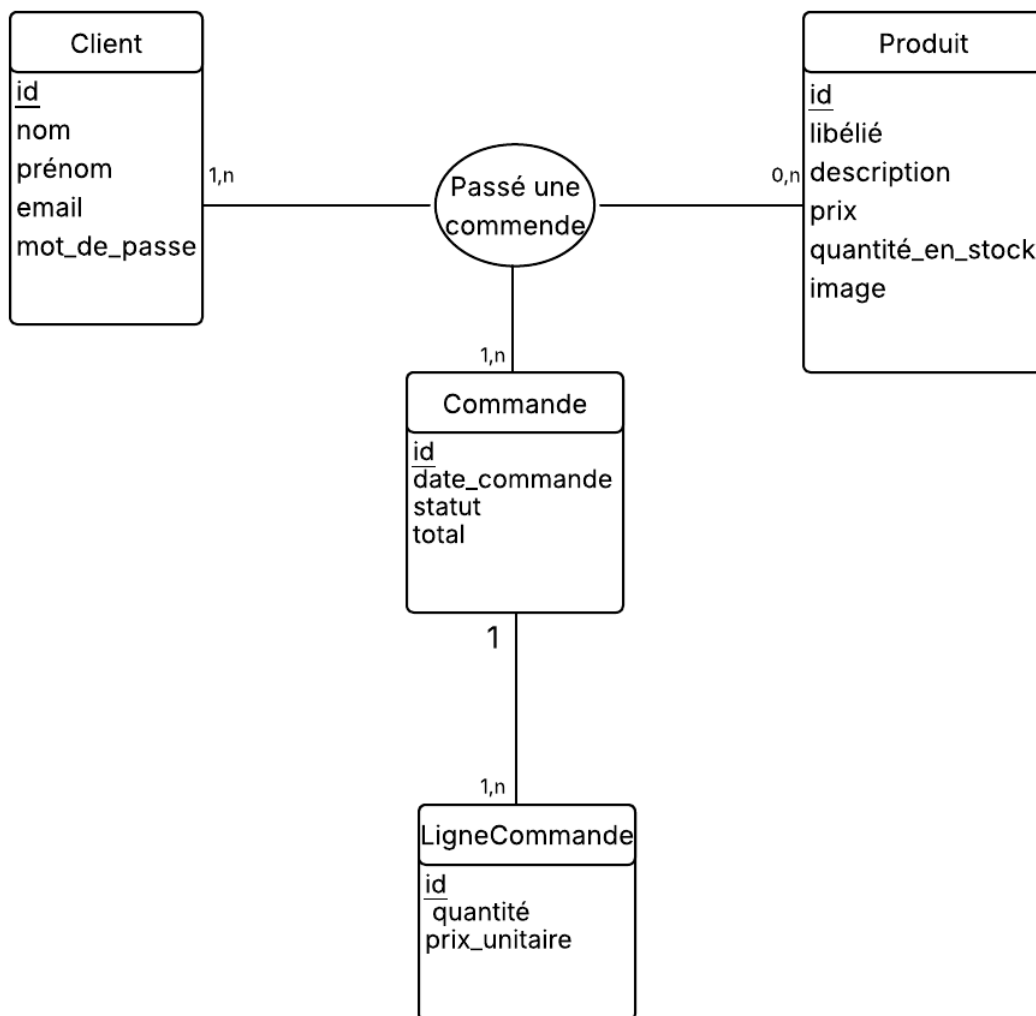
#### 8.1.1- Dictionnaire de Données (DD) :

| Entité      | Attribut | Type de données | Description                         | Contraintes                    |
|-------------|----------|-----------------|-------------------------------------|--------------------------------|
| Utilisateur | id       | INT             | Identifiant unique de l'utilisateur | Clé primaire, auto-incrémentée |
|             | nom      | VARCHAR(50)     | Nom de famille de l'utilisateur     | NOT NULL                       |
|             | prénom   | VARCHAR(50)     | Prénom de l'utilisateur             | NOT NULL                       |

|                 |                   |               |                                                   |                                    |
|-----------------|-------------------|---------------|---------------------------------------------------|------------------------------------|
|                 | email             | VARCHAR(100)  | Adresse email de l'utilisateur                    | NOT NULL, UNIQUE                   |
|                 | mot_de_passe      | VARCHAR(255)  | Mot de passe haché                                | NOT NULL                           |
|                 | date_naissance    | DATE          | Date de naissance de l'utilisateur                | Optionnel                          |
|                 | date_création     | DATETIME      | Date de création du compte                        | Par défaut : CURRENT_TIMESTAMP     |
| <b>Produit</b>  | id                | INT           | Identifiant unique du produit                     | Clé primaire, auto-incrémentée     |
|                 | titre             | VARCHAR(100)  | Titre ou nom du produit                           | NOT NULL                           |
|                 | description       | TEXT          | Description du produit                            | Optionnel                          |
|                 | prix              | DECIMAL(10,2) | Prix unitaire                                     | NOT NULL, $\geq 0$                 |
|                 | quantité_en_stock | INT           | Quantité disponible                               | NOT NULL, $\geq 0$                 |
|                 | image             | VARCHAR(255)  | Chemin ou URL de l'image (format PNG, JPG)        | Optionnel                          |
| <b>Commande</b> | id                | INT           | Identifiant unique de la commande                 | Clé primaire, auto-incrémentée     |
|                 | utilisateur_id    | INT           | Référence à l'utilisateur ayant passé la commande | Clé étrangère vers Utilisateur(id) |
|                 | date_commande     | DATETIME      | Date et heure de la commande                      | NOT NULL                           |
|                 | statut            | VARCHAR(50)   | État de la commande                               | NOT NULL                           |

|                      |               |               |                                            |                                 |
|----------------------|---------------|---------------|--------------------------------------------|---------------------------------|
|                      |               |               | (ex. : en cours, expédiée, livrée...)      |                                 |
|                      | total         | DECIMAL(10,2) | Montant total de la commande               | NOT NULL                        |
| <b>LigneCommande</b> | id            | INT           | Identifiant unique de la ligne de commande | Clé primaire, auto-incrémentée  |
|                      | commande_id   | INT           | Référence à la commande                    | Clé étrangère vers Commande(id) |
|                      | produit_id    | INT           | Référence au produit                       | Clé étrangère vers Produit(id)  |
|                      | quantité      | INT           | Quantité commandée du produit              | NOT NULL, $\geq 1$              |
|                      | prix_unitaire | DECIMAL(10,2) | Prix unitaire au moment de la commande     | NOT NULL                        |

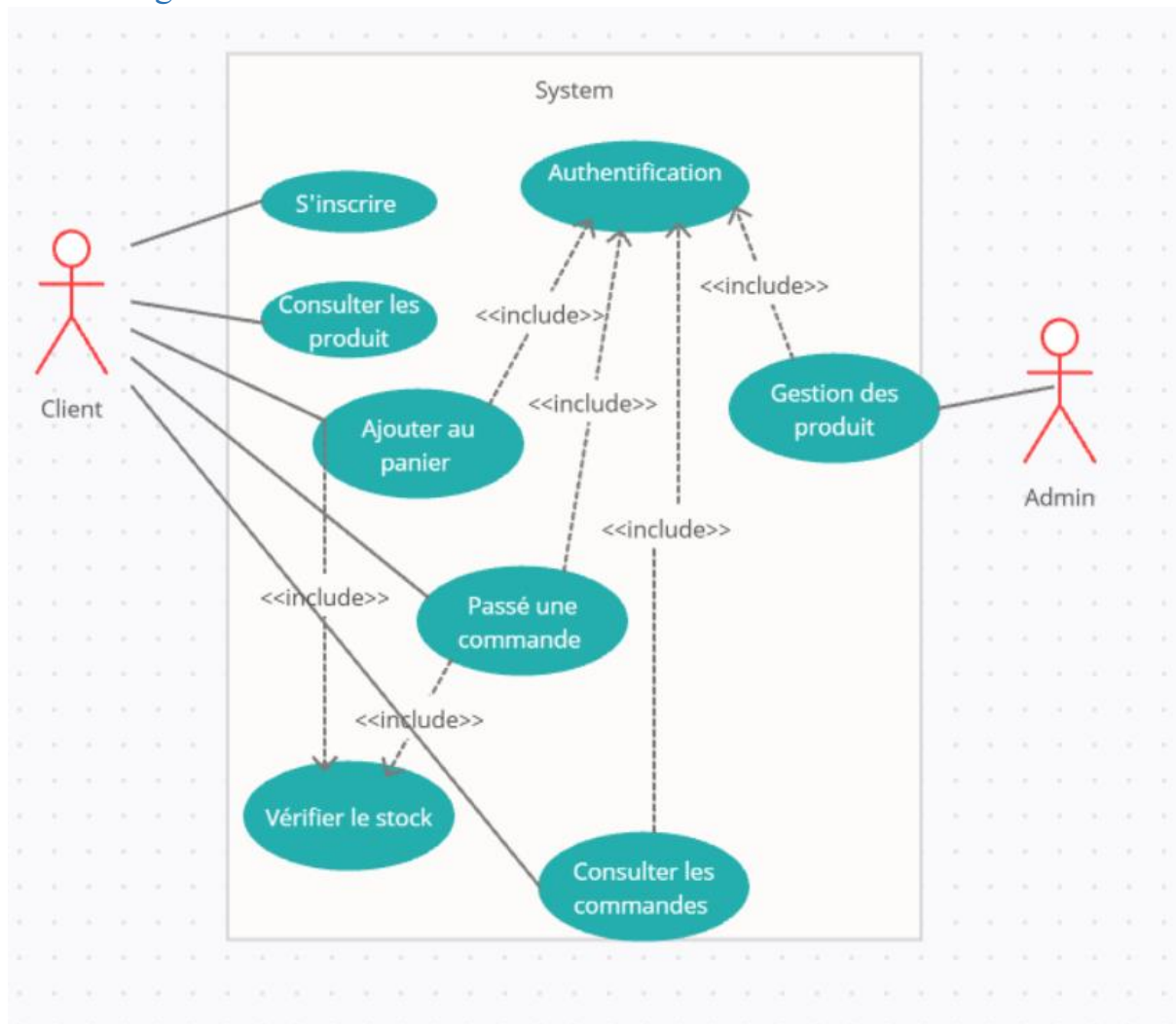
### 8.1.2- Modèle Conceptuel de Données (MCD):



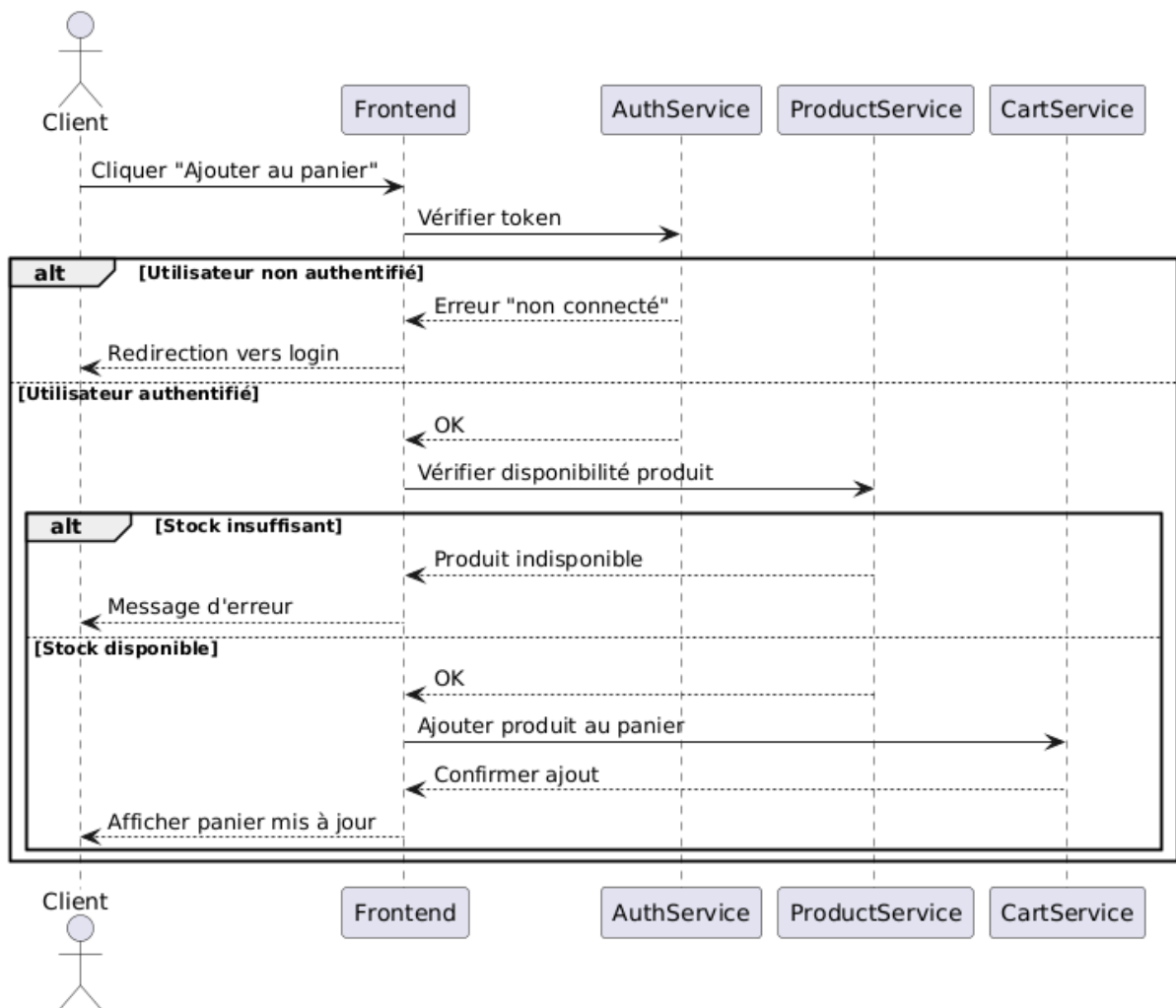
### 8.1.3- Le schéma relationnel de Données :

|                                                                       |
|-----------------------------------------------------------------------|
| UTILISATEUR(id, nom, prénom, email, mot_de_passe, date_naissance)     |
| PRODUIT(id, titre, description, prix, quantité_en_stock, image)       |
| COMMANDE(id, date_commande, statut, total, #id_utilisateur)           |
| LIGNECOMMANDE(id, quantité, prix_unitaire, #id_commande, #id_produit) |

#### 8.1.4- Diagramme de use case :



### 8.1.5 Diagramme de séquence – Ajouter au panier :



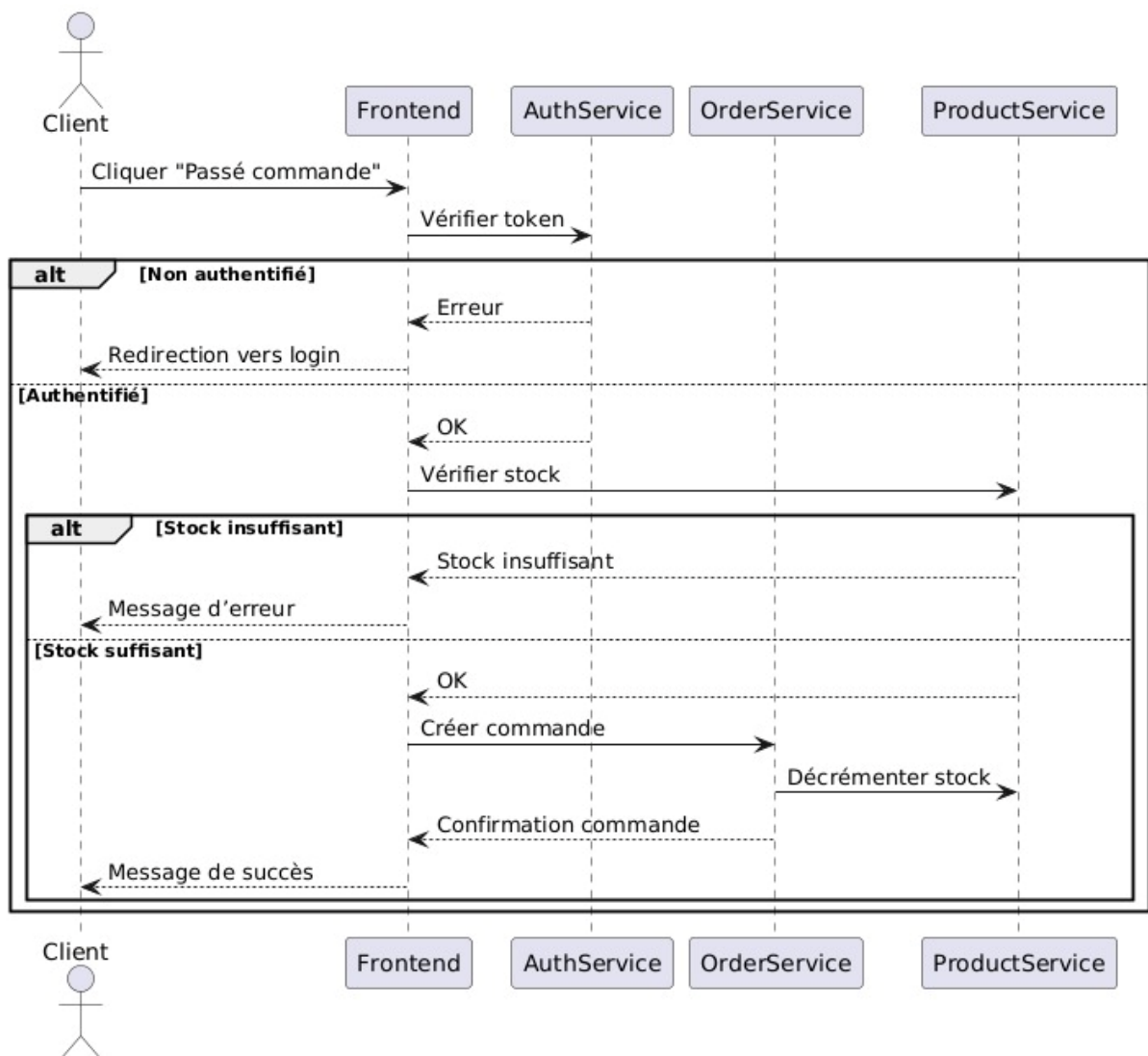
#### Description :

Ce diagramme illustre le scénario d'ajout d'un produit au panier par un client connecté.

#### Déroulement :

1. Le client clique sur "Ajouter au panier" sur l'interface frontend.
2. Le frontend envoie une requête au service d'authentification pour valider le token JWT.
3. Si l'utilisateur **n'est pas connecté**, il est redirigé vers la page de login.
4. Si l'utilisateur **est connecté**, le frontend interroge ProductService pour vérifier la disponibilité du produit.
5. Si le stock est insuffisant, un message d'erreur est affiché.
6. Si le stock est suffisant, le produit est ajouté au panier via CartService.
7. Le panier est mis à jour et une confirmation est affichée à l'utilisateur.

### 8.1.6 Diagramme de séquence – Passer une commande :



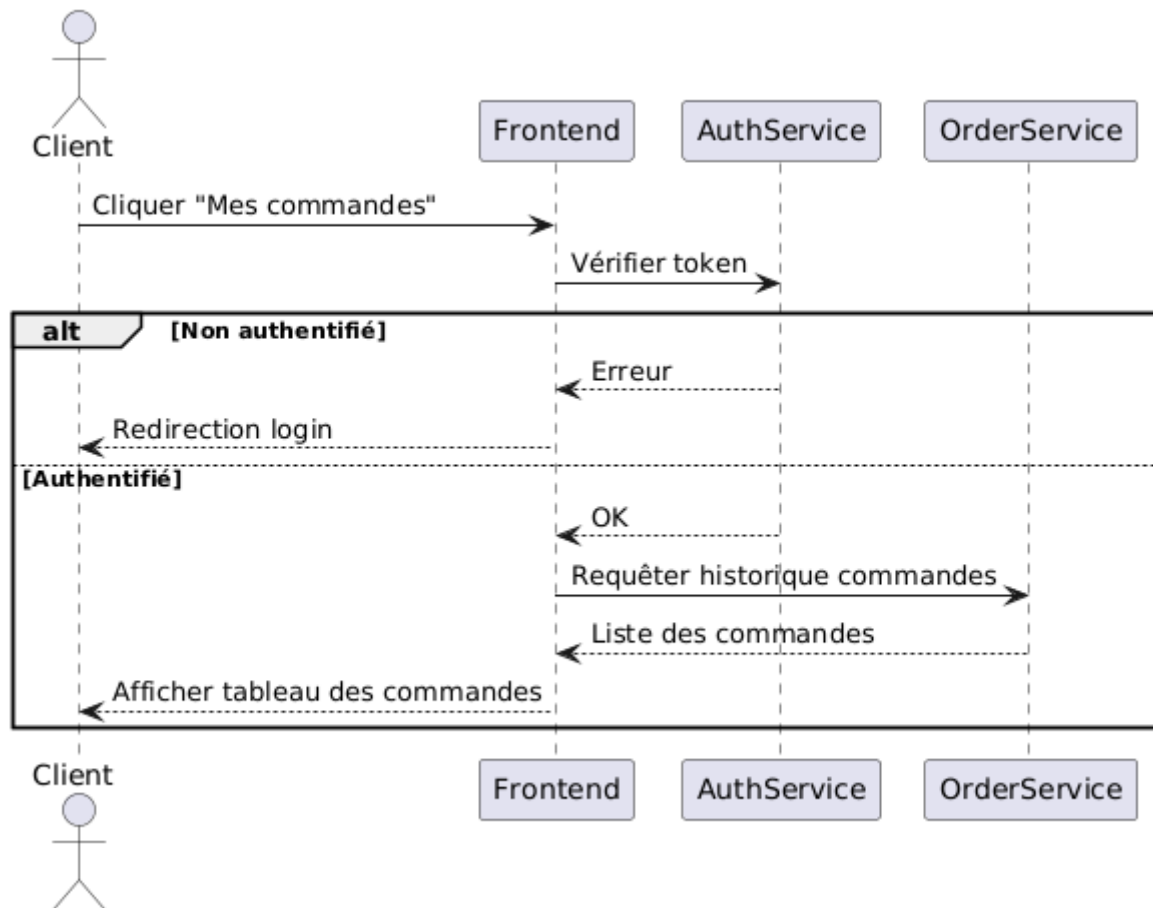
Ce diagramme décrit le processus de validation d'une commande par un utilisateur connecté.

#### Déroulement :

1. Le client valide son panier en cliquant sur "Passer commande".
2. Le système vérifie d'abord l'authentification via AuthService.
3. Si l'utilisateur n'est pas connecté, il est redirigé vers la page d'authentification.
4. Si l'utilisateur est authentifié :
  - Le frontend appelle ProductService pour vérifier que les produits en panier sont encore disponibles.
  - Si le stock est insuffisant, la commande est annulée avec un message d'erreur.
  - Si le stock est disponible :
    - Le frontend envoie la commande à OrderService.

- OrderService décrémente le stock via ProductService.
- Une confirmation de commande est retournée à l'utilisateur.

### 8.1.7 Diagramme de séquence – Consulter les commandes :

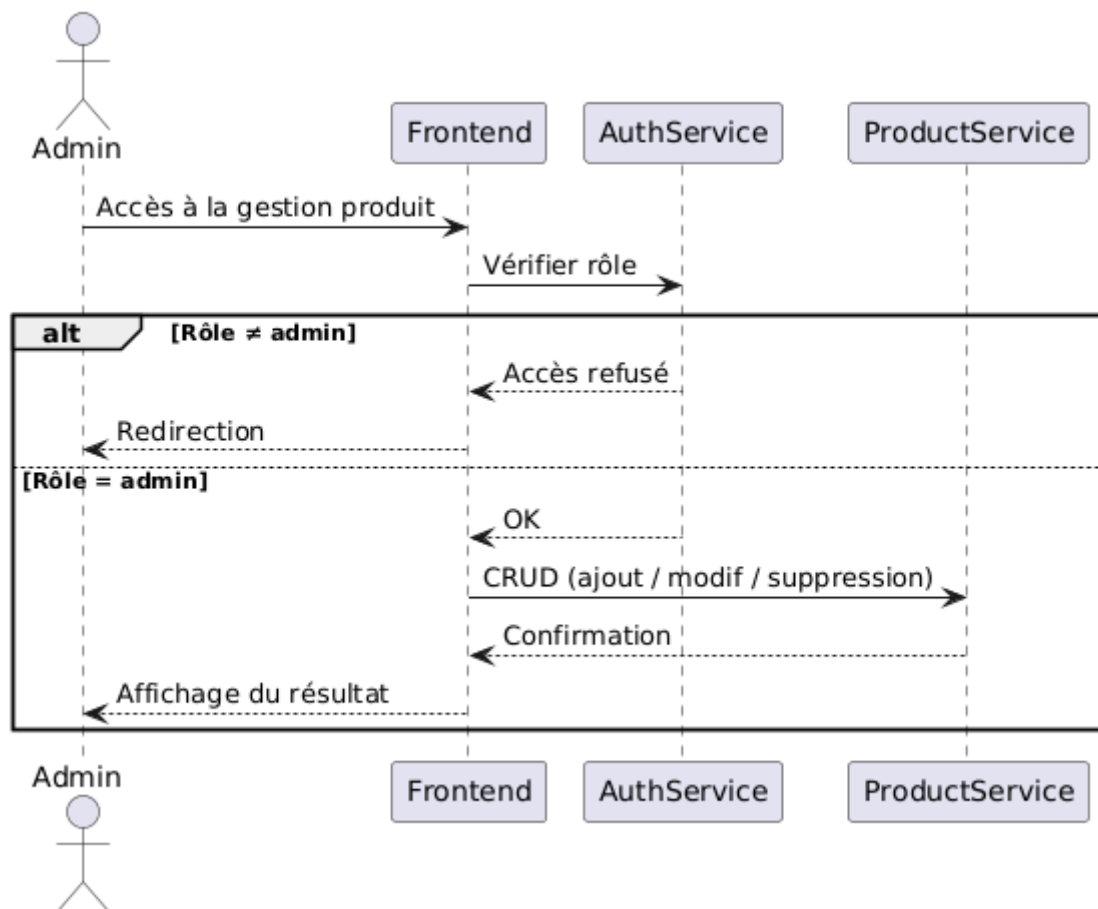


Ce diagramme représente l'action d'un client qui souhaite consulter l'historique de ses commandes.

#### Déroulement :

1. Le client clique sur "Mes commandes" dans le frontend.
2. Le système vérifie l'authentification de l'utilisateur.
3. Si l'utilisateur n'est pas connecté, il est redirigé.
4. Si connecté :
  - Le frontend envoie une requête au OrderService.
  - Ce dernier retourne la liste des commandes du client.
  - Le frontend affiche ces informations à l'utilisateur.

### 8.1.8 Diagramme de séquence – Gérer les produits (Admin) :

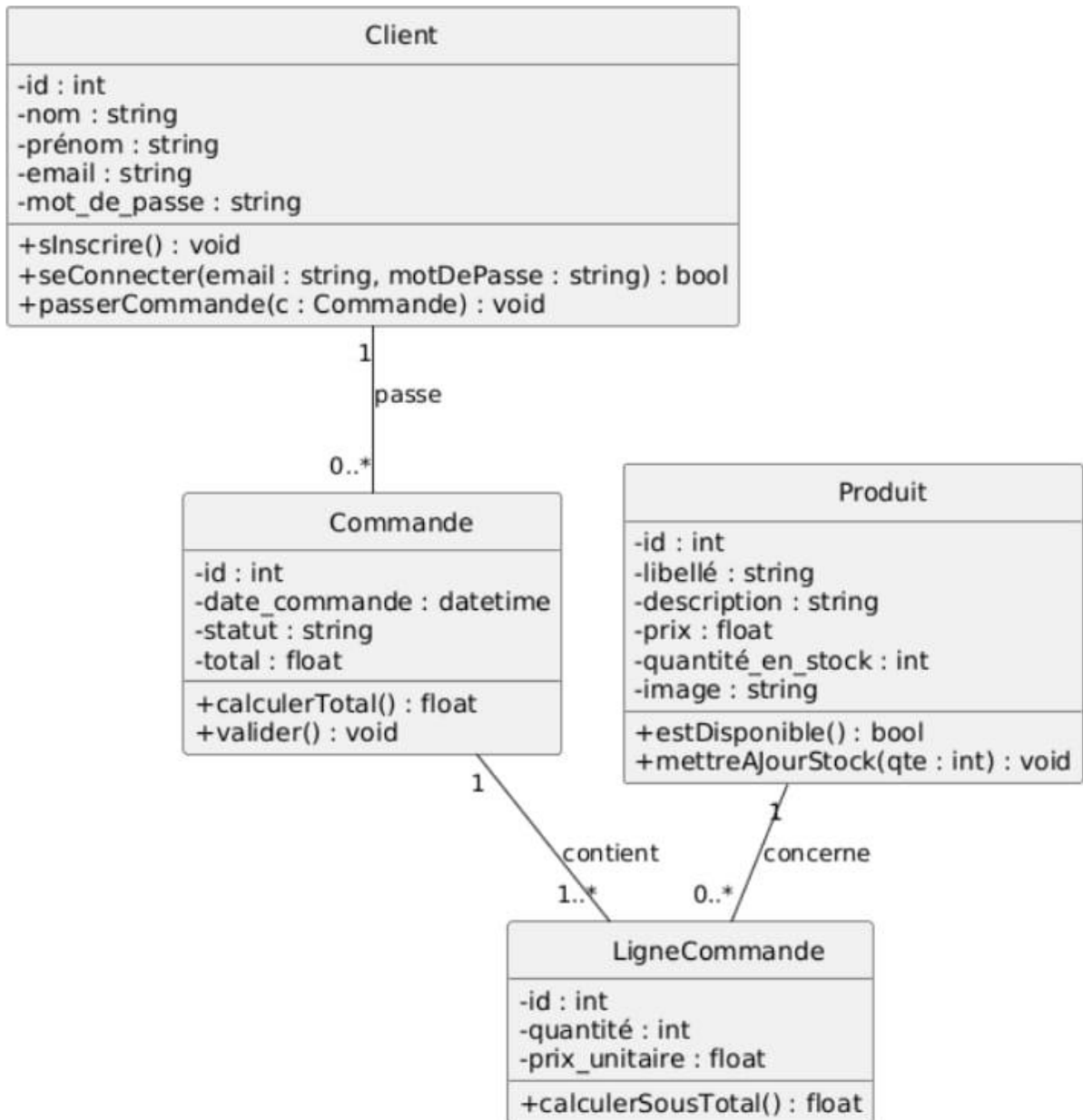


Ce diagramme décrit les interactions nécessaires à un administrateur pour effectuer des actions de gestion des produits.

#### Déroulement :

1. L'administrateur accède à l'espace "Gestion des produits".
2. Le frontend vérifie son rôle via AuthService.
3. Si l'utilisateur **n'est pas admin**, l'accès est refusé.
4. Si le rôle est valide :
  - L'administrateur peut envoyer des requêtes à ProductService pour :
    - Ajouter un nouveau produit
    - Modifier un produit existant
    - Supprimer un produit
  - ProductService traite la requête et retourne une confirmation.
  - Le frontend affiche une notification de succès ou d'échec.

### 8.1.9 Diagramme de classe :



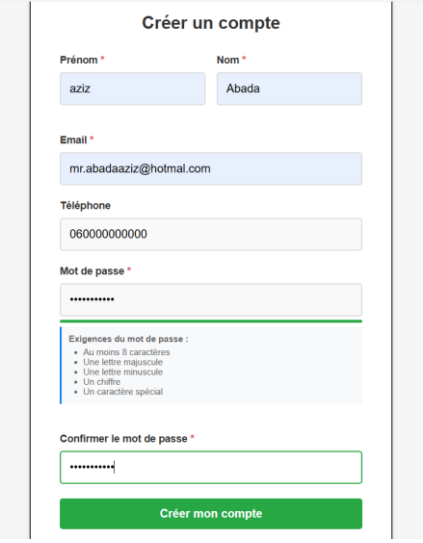
## 8.2- Modélisation :

### 8.2.1- page S'inscrire



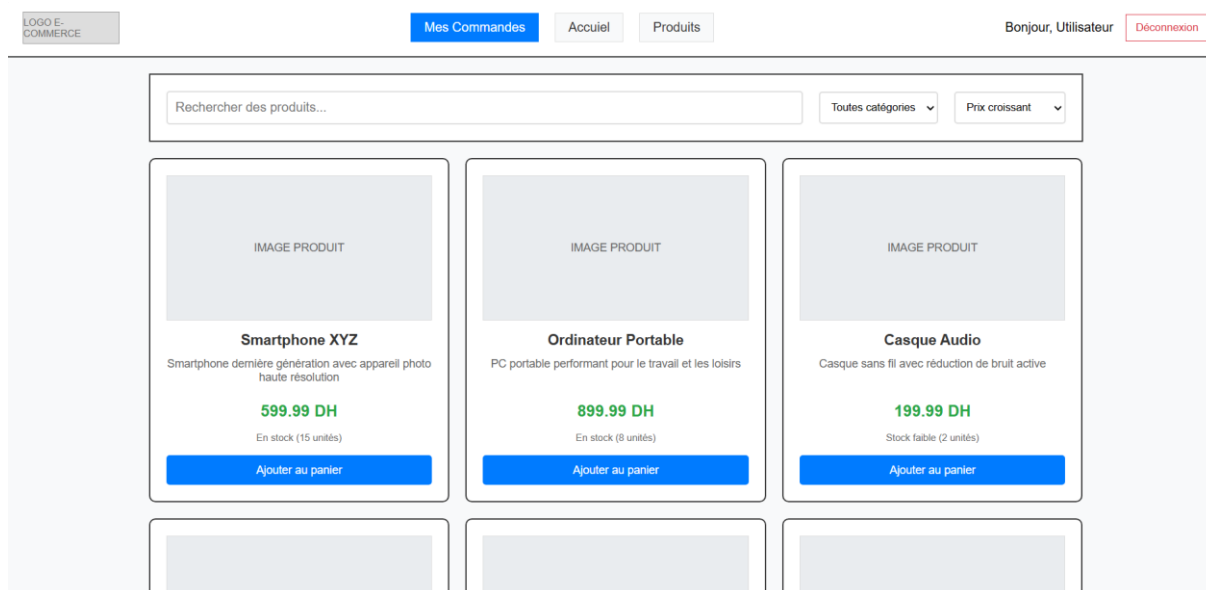
The screenshot shows a login form titled "Connexion". At the top, there is a "LOGO" placeholder. Below the title, there are two input fields: "Email" with the placeholder text "votre@email.com" and "Mot de passe" with masked characters. A blue button labeled "Se connecter" is positioned below the password field. Underneath the button, there is a link "Mot de passe oublié ?". At the bottom of the form, there is a link "Pas encore de compte ? S'inscrire".

### 8.2.2- page connexion



The screenshot shows a registration form titled "Créer un compte". It contains several input fields: "Prénom \*" with the value "aziz", "Nom \*" with the value "Abada", "Email \*" with the value "mr.abadaaziz@hotmail.com", "Téléphone" with the value "06000000000", and "Mot de passe \*" with masked characters. Below the password field, there is a section titled "Exigences du mot de passe :" with a list of requirements: "Au moins 8 caractères", "Une lettre majuscule", "Une lettre minuscule", "Un chiffre", and "Un caractère spécial". Below this, there is a field for "Confirmer le mot de passe \*" with masked characters. At the bottom, there is a green button labeled "Créer mon compte".

## 8.2.3- page des produits



## 8.2.4- page connexion

